University of Chittagong

Department of Computer Science and Engineering

Healthcare Web Application

D1: Requirement Document

CSE-616: Web Engineering Lab

Submitted to,

Rokan Uddin Faruqui

Professor

Department of Computer Science and Engineering

University of Chittagong

Chittagong-4331, Bangladesh

Submitted by,

Group - 11		
Sl.	**Name**	**Student ID**
1	Shahrukh Ahmed Meherab	22701006
2	Md. Emranul Hoque	22701038
3	Abu Ruhan Mahamud	22701074

Date of Submission: September 17, 2025

Contents

1 Project Overview **3**

2 Functional Requirements **3**

2.1 User Authentication and Login 3

2.2 Patient Features 3

2.3 Doctor Features 4

2.4 Administrator Features 4

2.5 Additional Features 5

3 Non-Functional Requirements **5**

3.1 Security 5

3.2 Performance 6

3.3 Reliability 6

4 User Stories with Acceptance Criteria **6**

4.1 Patient-User Stories 6

4.1.1 Story 1: Account Creation 6

4.1.2 Story 2: Appointment Booking 7

4.1.3 Story 3: Medical History Access 7

4.2 Doctor-User Stories 7

4.2.1 Story 1: Patient Record Access 7

4.2.2 Story 2: Schedule Management 8

4.2.3 Story 3: Prescription Writing 8

4.3 Administrator-User Stories 8

4.3.1 Story 1: User Management 8

4.3.2 Story 2: System Monitoring 9

5 Technology Stack Justification **9**

5.1 Backend: Django (Python) 9

5.2 Frontend: React.js 10

5.3 Database: PostgreSQL 10

6 Team Roles and Responsibilities **10**

6.1 Team Lead: Abu Ruhan Mahamud 10

6.2 Backend Developer: Md. Emranul Hoque 11

6.3 Frontend Developer: Shahrukh Ahmed Meherab 11

7 Project Timeline and Milestones **11**

7.1 Major Deliverables 11

8 Risk Assessment and Mitigation Strategies **12**
8.1 Technical Risks 12
8.2 Project Management Risks 13
8.3 Infrastructure Risks 13
8.4 Risk Response Plan 13

9 Project Setup and Management Tools **14**
9.1 Git Repository Initialization 14
9.2 Trello Board Setup 14
9.3 Slack Workspace Configuration 14

1 Project Overview

The main goal of the Healthcare Web Application is to make healthcare management easier by letting people access medical records, schedule appointments, and communicate with healthcare providers. The application connects patients, doctors, and administrators in one structured system.
The system has three types of users:

- **Patients** - manage their health information and book appointments
- **Doctors** - view patient records and manage their schedule
- **Administrators** - manage user accounts and system settings

We will use agile development over 12 weeks to build the app in small steps and receive feedback regularly to make sure we are on the right track.

2 Functional Requirements

2.1 User Authentication and Login

The system needs secure login for three types of users: patients, doctors, and administrators. Each user type can only access the parts of the system they need. Key features include,

- User registration with email confirmation
- Secure login and logout
- Password reset through email
- Account management (update profile, change password)
- Different access levels for each user type

2.2 Patient Features

1. **Profile Management**
    - Create and update personal information
    - Add emergency contact details
    - Manage basic medical information
2. **Medical History**

- View past appointments and treatments
- See prescription history
- Access test results and doctor notes

3. **Appointment Booking**
   - View available doctors and time slots
   - Book appointments online
   - Cancel or reschedule appointments
   - Get appointment reminders

2.3 Doctor Features

1. **Patient Management**
   - View list of assigned patients
   - Access patient medical history
   - Review upcoming appointments
2. **Schedule Management**
   - See daily and weekly appointment schedule
   - Update appointment status
   - Add notes after patient visits
3. **Prescription Management**
   - Create new prescriptions
   - Update existing medications
   - View patient's current prescriptions

2.4 Administrator Features

1. **User Management**
   - Create, edit, and deactivate user accounts
   - Assign roles to users (patient, doctor, admin)
   - Reset user passwords when needed
2. **System Monitoring**

- View basic usage statistics
- Monitor system activity
- Generate simple reports

3. **System Settings**
    - Configure appointment time slots
    - Set up system notifications
    - Manage basic application settings

2.5 Additional Features

1. **Medical imaging upload and viewing**
    - Medical documents(PDF, images) can be uploaded to the system
    - Doctors can view uploaded files securely
2. **Medication tracking and reminders**
    - Reminder system for taking medications
    - Display current prescriptions
    - Notification for refills

3 Non-Functional Requirements

3.1 Security

The application must protect patient information and prevent unauthorized access:

- All data sent between browser and server is encrypted (HTTPS)
- User passwords are stored securely
- Input validation to prevent common attacks
- Session management with automatic logout
- Basic protection of sensitive patient data

3.2 Performance

The system should work well under normal use:

- Pages load within reasonable time
- Support multiple users at the same time
- Basic database optimization for faster searches
- Efficient file uploads and downloads

3.3 Reliability

The system should work consistently:

- Basic error handling with helpful messages
- Regular data backups
- System logging for troubleshooting
- Stable operation under normal conditions

4 User Stories with Acceptance Criteria

4.1 Patient-User Stories

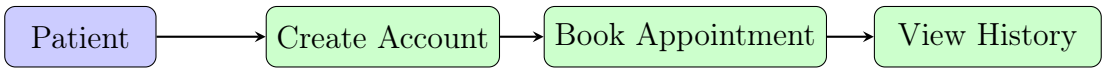

Figure 1: Patient-User Journey

4.1.1 Story 1: Account Creation

"As a patient, I want to create an account so I can use the healthcare system."

Acceptance Criteria:

- Registration form with required fields works correctly
- Email verification is sent and works
- User can log in with new account
- Profile information can be updated after registration

4.1.2 Story 2: Appointment Booking

"As a patient, I want to schedule appointments with doctors online."

Acceptance Criteria:

- Available time slots are shown clearly
- Booking process is simple and works
- Confirmation is sent after booking
- Appointments appear in my schedule

4.1.3 Story 3: Medical History Access

"As a patient, I want to see my medical history to track my health."

Acceptance Criteria:

- Medical history is displayed in chronological order
- Information is easy to read and understand
- Past prescriptions and treatments are shown
- Data loads quickly when requested

4.2 Doctor-User Stories

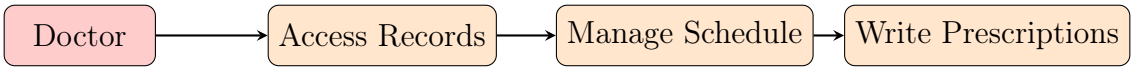

Figure 2: Doctor-User Journey

4.2.1 Story 1: Patient Record Access

"As a doctor, I need to view patient records to provide good care."

Acceptance Criteria:

- Patient list is easy to navigate
- Complete medical history loads properly
- Current medications are clearly displayed
- Information is organized and readable

4.2.2 Story 2: Schedule Management

"As a doctor, I want to manage my appointments efficiently."

Acceptance Criteria:

- Daily and weekly calendar views work
- Patient information is easily accessible
- Appointment details are clear
- Schedule updates work properly

4.2.3 Story 3: Prescription Writing

"As a doctor, I need to create prescriptions for patients."

Acceptance Criteria:

- Prescription form is easy to use
- Patient medication history is available
- New prescriptions are saved correctly
- Prescription list updates properly

4.3 Administrator-User Stories

Figure 3: Administrator-User Journey

4.3.1 Story 1: User Management

"As an administrator, I need to manage user accounts for security."

Acceptance Criteria:

- All user accounts are displayed with their roles
- New accounts can be created successfully
- Account information can be edited
- Users can be deactivated when needed

4.3.2 Story 2: System Monitoring

"As an administrator, I want to monitor system usage."

Acceptance Criteria:

- Basic usage statistics are displayed
- User activity can be reviewed
- Simple reports can be generated
- Data is presented clearly

5 Technology Stack Justification

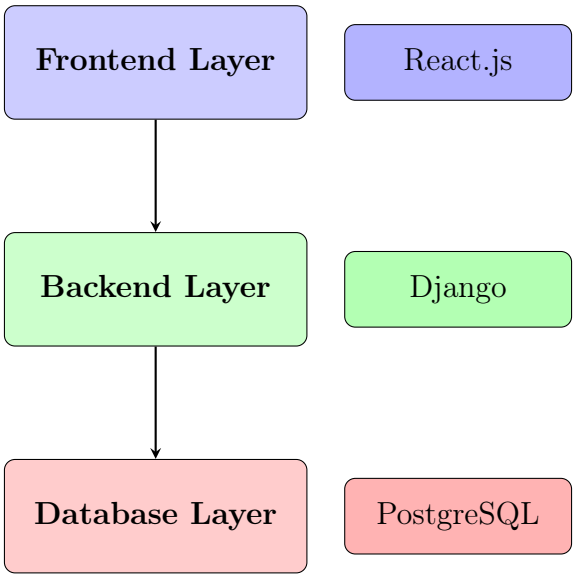

Figure 4: Technology Stack Architecture

5.1 Backend: Django (Python)

We chose Django because:

- Built-in security features protect against common web attacks
- User authentication system works well for our three user types
- Good documentation and community support
- Handles database relationships effectively
- Django REST Framework makes API development easier

5.2 Frontend: React.js

We chose React because:

- Component-based structure fits our different user interfaces
- Good for interactive features like appointment booking
- Large community with helpful resources
- Works well with Django APIs
- Team members have some experience with React

5.3 Database: PostgreSQL

We chose PostgreSQL because:

- Reliable and handles complex data relationships
- Good performance for our expected usage
- Strong backup and recovery features
- Free and open-source
- Works well with Django

Our selected stack of Django, React, and PostgreSQL offers a coherent and practical solution for the healthcare application. Django provides a secure and structured backend with built-in authentication and ORM support, React delivers responsive and interactive interfaces for different user roles, and PostgreSQL ensures reliable handling of complex healthcare data. This combination not only supports our functional requirements such as appointment scheduling, prescription management, and medical record access, but also addresses non-functional needs like security, scalability, and performance. The technologies complement each other well, align with the project guidelines, and match the team's skill set, allowing us to build a system that is both robust and achievable within the semester timeline.

6 Team Roles and Responsibilities

6.1 Team Lead: Abu Ruhan Mahamud

Primary Responsibilities:

- Coordinate project activities and deadlines

- Manage communication between team members
- Plan sprint activities and track progress
- Handle project documentation and submissions
- Resolve conflicts and make final decisions

6.2 Backend Developer: Md. Emranul Hoque

Primary Responsibilities:

- Develop Django application and APIs
- Design and implement database models
- Handle user authentication and security
- Set up deployment infrastructure

6.3 Frontend Developer: Shahrukh Ahmed Meherab

Primary Responsibilities:

- Build React user interfaces for all user types
- Implement responsive design for different screen sizes
- Connect frontend to backend APIs
- Handle user experience and interface design
- Test frontend functionality across browsers

7 Project Timeline and Milestones

7.1 Major Deliverables

Timeline	Deliverable	Description
Week 2 (Sep 17, 2025)	Requirements & Setup	Complete requirements document, set up development environment, create project repository
Week 4 (Oct 5, 2025)	Design Documentation	Database design, API specifications, UI mockups, system architecture

Timeline	Deliverable	Description
Week 6 (Oct 19, 2025)	Backend Implementation	User authentication, core database models, basic API endpoints, testing framework
Week 8 (Nov 2, 2025)	Frontend Integration	User interfaces for all user types, API integration, responsive design
Week 10–12 (Nov 16–30, 2025)	Final Development	Additional features, optimization, testing, deployment, documentation

8 Risk Assessment and Mitigation Strategies

8.1 Technical Risks

Risk	Impact	Probability	Mitigation
Learning curve with Django/React	Medium	Medium	Focus on core features first, allow extra time for practice, rely on tutorials and documentation.
Frontend–Backend integration issues	High	Medium	Define API endpoints early, test small parts often, and keep regular communication.
Database performance issues	Medium	Low	Plan schema carefully, test with realistic data, optimize queries if needed.

8.2 Project Management Risks

Risk	Impact	Probability	Mitigation
Scope creep	High	Medium	Stick to agreed features, record changes, prioritize core functionality.
Team availability	High	Low	Share knowledge across members, maintain clear documentation, plan backup responsibilities.
Missing deadlines	High	Medium	Track progress weekly, identify problems early, and adjust scope if delays occur.

8.3 Infrastructure Risks

Risk	Impact	Probability	Mitigation
Security vulnerabilities	High	Medium	Use Django's built-in security features, follow best practices, check code for common issues.
Data loss	High	Low	Take regular backups, use Git carefully, keep multiple copies of important data.
Deployment problems	Medium	Medium	Test deployment steps early, document the process, and prepare backup options.

8.4 Risk Response Plan

- **High-impact risks:** Address immediately with all team resources.
- **Medium-impact risks:** Monitor closely and prepare clear response steps.
- **Low-impact risks:** Keep awareness but avoid over-planning.
- **Weekly review:** Update risk list and mitigation status during team meetings.
- **Communication:** Report serious risks to the instructor early rather than waiting.

9 Project Setup and Management Tools

As part of Deliverable 1 (D1), we have established the essential project management infrastructure and development tools for our project.

9.1 Git Repository Initialization

Backend Repository:

- **Repository URL:** https://github.com/Abu-Ruhan-Mahamud/Web_Engineering_Backend

Frontend Repository:

- **Repository URL:** https://github.com/Abu-Ruhan-Mahamud/Web_Engineering_Frontend

9.2 Trello Board Setup

- **Board URL:** https://webengineeringg.slack.com

9.3 Slack Workspace Configuration

- **Workspace URL:** https://trello.com/invite/68cacc52cc06db973a406208/ATTI8ee8b57123f3349fe4aae2e4a04fc881241AFA87